
Fundamentos de las redes neuronales convolucionales

PID_00290596

Jordi Casas Roma
Anna Bosch Rue

Tiempo mínimo de dedicación recomendado: 2 horas



Universitat
Oberta
de Catalunya

**Jordi Casas Roma**

Doctor en Informática (Universitat Autònoma de Barcelona, UAB, 2014), Máster en Inteligencia Artificial Avanzada (Universidad Nacional de Educación a Distancia, UNED, 2011) y Licenciado en Informática (Universitat Autònoma de Barcelona, UAB, 2002). Actualmente, es profesor agregado de la Facultad de Informática, Multimedia y Telecomunicaciones de la Universitat Oberta de Catalunya (UOC). Sus actividades docentes se concentran principalmente en minería de datos, aprendizaje automático, *big data* y minería de gráficos. Es el director del máster en Ciencia de datos (Data Science) de la Universitat Oberta de Catalunya (UOC). Sus principales ámbitos de investigación incluyen la inteligencia artificial, el aprendizaje automático, minería de grafos y la privacidad de datos. Desde 2020 es líder de grupo en ADaS Lab (Applied Data Science Lab) donde investiga la aplicación de algoritmos de inteligencia artificial a problemas del mundo real.

El encargo y la creación de este recurso de aprendizaje UOC han sido coordinados por el profesor: Jordi Casas Roma

Cómo citar este recurso de aprendizaje con el estilo Harvard:

Casas Roma, J., Bosch Rue, A. (2023) *Fundamentos de las redes neuronales convolucionales*. [Recurso de aprendizaje textual]. 1ª edición. Barcelona: Fundació Universitat Oberta de Catalunya (FUOC).

Primera edición: febrero 2024

© de esta edición, Fundació Universitat Oberta de Catalunya (FUOC)

Av. Tibidabo, 39-43, 08035 Barcelona

Autoría: Jordi Casas Roma, Anna Bosch Rue

Producción: FUOC

Todos los derechos reservados

Ninguna parte de esta publicación, incluyendo el diseño general y la cubierta, puede ser copiada, reproducida, almacenada o transmitida de ninguna forma ni por ningún medio, sea este eléctrico, químico, mecánico, óptico, grabación, fotocopia, o cualquier otro, sin la previa autorización escrita de los titulares de los derechos.

Índice

Introducción	5
Objetivos	6
1. Contextualización	7
2. Conceptos preliminares	10
2.1. La operación de convolución	10
2.2. Ventajas derivadas de la convolución	13
2.2.1. Interacciones o conexiones dispersas	13
2.2.2. Compartición de parámetros	14
2.3. Consideraciones finales	16
3. Componentes de una red neuronal convolucional	17
3.1. Capa de convolución	17
3.1.1. Filtros o <i>kernel</i>	17
3.1.2. Valor de relleno (<i>padding</i>)	19
3.1.3. Convoluciones por pasos	20
3.2. Capa de agrupamiento	22
3.3. Capa totalmente conectada	23
3.4. Capa de activación	25
3.5. Capa de <i>dropout</i>	26
4. Estructura de una red neuronal convolucional	27
4.1. Estructura básica	27
4.2. Ejemplo de red neuronal convolucional	30
Resumen	34
Glosario	35
Bibliografía	36

Introducción

Iniciaremos este módulo didáctico con una breve contextualización de las redes neuronales convolucionales y algunas de sus aplicaciones «clásicas», es decir, los dominios o aplicaciones donde las redes neuronales convolucionales se aplicaron en primera instancia. Actualmente, vemos aplicaciones relacionadas con redes neuronales convolucionales en una gran cantidad de sectores, dominios o aplicaciones, que incluyen temas tan diversos como la medicina, la agricultura o la astronomía, por poner algunos ejemplos.

Seguidamente, en el segundo capítulo de este texto, introduciremos la operación de convolución, que es el núcleo de las redes neuronales convolucionales. Es importante entender esta operación básica, así como conocer las ventajas que ofrece y sus posibles limitaciones.

Aunque las capas convolucionales, formadas por neuronas que aplican la operación de convolución, son la base de las redes neuronales convolucionales, no son el único tipo de capa que aparece en este tipo de redes. Es muy habitual, y generalmente necesario, el empleo de capas de agrupamiento, capas de activación, capas completamente conectadas y capas de *dropout* para que la red pueda efectuar las operaciones deseadas y conseguir un buen comportamiento en diferentes entornos o dominios. Esta discusión se desarrollará en el tercer capítulo de este módulo didáctico.

Finalmente, en el cuarto y último capítulo, trataremos la estructura básica de una red neuronal convolucional y veremos un ejemplo concreto de una red inspirada en LeNet-5 y desarrollada para el reconocimiento de dígitos escritos manualmente.

Objetivos

En los materiales didácticos de este módulo encontraremos las herramientas indispensables para asimilar los siguientes objetivos:

1. Entender la operación de convolución, así como sus principales ventajas e inconvenientes.
2. Conocer el funcionamiento y los objetivos de la capa de convolución.
3. Conocer las otras capas empleadas, generalmente, en las redes neuronales convolucionales.
4. Comprender la estructura básica de una red neuronal convolucional.

1. Contextualización

Iniciaremos este módulo didáctico dedicado a las redes neuronales convolucionales (en inglés, *Convolutional Neural Networks*, CNN) viendo algunos ejemplos de aplicaciones de estos modelos en entornos reales y cotidianos que todos conocemos.

Las redes convolucionales son un tipo especial de redes neuronales para procesar datos con tipología cuadrículada, como las imágenes. La visión por ordenador es una de las áreas que ha avanzado más rápidamente gracias al *deep learning*.

Los modelos de *deep learning* en visión por ordenador están ayudando y mejorando muchos aspectos de nuestra vida cotidiana, como por ejemplo:

- Están ayudando a los vehículos autónomos (*self-driving cars*) a detectar dónde están los otros coches y personas a su alrededor para evitarlos y no colisionar con ellos.
- Están haciendo que el reconocimiento facial sea considerablemente mejor de lo que había sido hasta hace pocos años y han propiciado que podamos desbloquear nuestro teléfono móvil o incluso abrir las puertas de nuestra casa u oficina simplemente usando nuestra cara.
- Muchos de nosotros tenemos aplicaciones en el teléfono móvil que nos enseñan fotos de nuestro interés, clasifican las fotos que tomamos con nuestra cámara, hacen reconocimiento de objetos en ellas, etc. Muchas de las empresas que construyen estas aplicaciones usan modelos basados en *deep learning* para mostrarnos las imágenes más atractivas, más relevantes o más bonitas.

Hay dos razones principales por las que el *deep learning* aplicado a la visión por ordenador es realmente interesante: (1) primero, los avances tan rápidos en esta área permiten, y permitirán, crear aplicaciones que nos parecían imposibles hace pocos años; (2) la comunidad que trabaja en visión por ordenador ha demostrado ser muy creativa e innovadora en las arquitecturas y los algoritmos de *deep learning* que están usando y esto ha creado sinergias importantes en otras áreas científicas, como por ejemplo, en el reconocimiento del habla o los asistentes virtuales.

Lectura complementaria

Y. LeCun.
«Generalization and network design strategies». Technical Report CRG-TR-89-4, 1989.

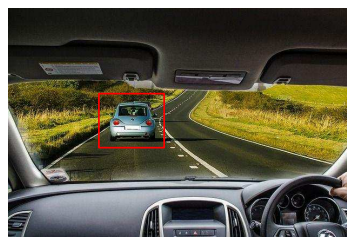
Algunos de los problemas de visión por ordenador que se han resuelto empleando *deep learning* incluyen:

- **Clasificación de imágenes**, también llamado reconocimiento de imágenes. Dada una imagen, como por ejemplo la figura 1 (a), queremos responder a la pregunta: ¿hay un coche en esta imagen?
- **Detección de objetos**. Si estamos construyendo un coche que conduce de forma autónoma, no solamente necesitamos saber si hay coches en la imagen, sino que necesitamos obtener la posición de los otros coches, por ejemplo dibujando cajas a su alrededor (figura 1 (b)) para no colisionar con ellos.
- **Transferencia de estilos**. Imaginemos que tenemos una imagen y la queremos pintar con un estilo diferente. En la transferencia neuronal de estilos, tenemos una imagen contenido (figura 2 (a)) y una imagen estilo (figura 2 (b)), por ejemplo un Van Gogh. La red neuronal, une las dos para repintar la imagen contenido empleando la imagen estilo y obteniendo la imagen resultado, que en este ejemplo podemos ver en la figura 2 (c).

Figura 1. Ejemplos de visión por ordenador que pueden resolverse mediante *deep learning*: (a) clasificación de imágenes: «¿hay un coche?» y (b) detección de objetos: localización de coches mediante una *bounding box*



(a) Clasificación



(b) Detección

Figura 2. Ejemplos de visión por ordenador que pueden resolverse mediante *deep learning*: imagen contenido (a) + imagen estilo (b) = imagen resultado (c)



(a) Contenido

+



(b) Estilo

=



(c) Resultado

Uno de los retos más importantes en visión por ordenador es que las entradas pueden ser realmente grandes. Imaginemos que tenemos imágenes de 128 píxeles de alto por 128 píxeles de ancho (128×128). Si son imágenes en color, la dimensión del vector de entrada es de $128 \times 128 \times 3 = 49.152$ píxeles, porque tenemos 3 canales de color, generalmente codificados empleando el modelo RGB. El valor obtenido, 49.152, no parece un vector muy grande para

Color RGB

RGB (de las siglas en inglés: *Red*, *Green* y *Blue*) es un modelo de color basado en la síntesis aditiva, con el que es posible representar un color mediante la mezcla por adición de los tres colores de luz primarios.

los ordenadores modernos, pero las imágenes de 128×128 son muy pequeñas. Imaginemos ahora que tenemos imágenes de $2048 \times 2048 \times 3 \approx 12 \times 10^6$. Es decir, necesitamos un vector de más de 12 millones de dimensiones para almacenar dicha imagen. Si en una red neuronal la primera capa oculta (*hidden layer*) tiene 2.048 neuronas (*hidden units*), entonces la dimensión de la primera matriz será de $2048 \times 12 \times 10^6 \approx 25 \times 10^9$ posiciones. Si usamos una red conectada completamente (*fully-connected*), ¡esto implicaría que debemos aprender 25 mil millones de parámetros en la primera capa oculta! No hace falta decir que es poco práctico ya que, a parte de los problemas computacionales y de memoria que podamos tener para entrenar el modelo, es muy difícil tener suficientes datos para prevenir el sobreentrenamiento (*overfitting*) de la red.

Las imágenes de 128×128 se consideran de baja resolución y las de 2048×2048 de alta resolución. Cuando trabajamos en visión por ordenador no queremos trabajar con imágenes de baja resolución, por lo que si queremos usar imágenes más grandes debemos implementar algún método que nos permita reducir la dimensión de las imágenes conservando el máximo (si es posible, toda) la información que estas contienen. Es aquí donde aparece el concepto de **operación de convolución**, que es el bloque fundamental de las redes convolucionales. La convolución es una operación matemática y las redes convolucionales son redes neuronales que usan esta operación en lugar de la multiplicación de matrices en sus capas.

2. Conceptos preliminares

En este capítulo nos centraremos en la operación básica de las redes neuronales convolucionales (CNN), la convolución. Veremos sus fundamentos matemáticos y sus principales propiedades, que nos ayudarán a entender el funcionamiento de este tipo de redes neuronales.

2.1. La operación de convolución

La **convolución** es una operación matemática sobre dos funciones (f y g) que produce una tercera función (s) que expresa como la forma de una es modificada por la otra. Típicamente, la operación de convolución se denota con un asterisco (*):

$$s(t) = (f * g)(t) = \int f(\tau)g(t - \tau)d\tau \quad (1)$$

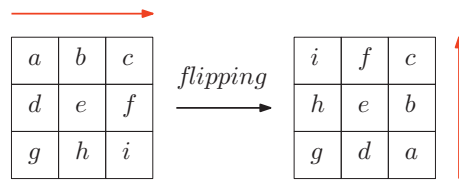
La operación de convolución se puede aplicar en distintas áreas y aplicaciones tales como probabilidad, estadística, visión por ordenador, procesamiento del lenguaje natural, imagen y procesamiento de la señal, ingeniería y ecuaciones diferenciales.

En la terminología de las redes convolucionales, el primer argumento (la f en la ecuación anterior) de la convolución se refiere a la **entrada (input)** y el segundo argumento (la función g) es el **filtro o kernel**. La salida de la función (s) se refiere al **mapa de características o feature map**. Normalmente, los datos usados por las redes convolucionales son datos discretos y, por lo tanto, usaremos la convolución discreta:

$$s(t) = (f * g)(t) = \sum_{\tau=-\infty}^{\infty} f(\tau)g(t - \tau) \quad (2)$$

En aplicaciones de aprendizaje automático (*machine learning*) la entrada es un vector multidimensional de datos y el *kernel* es un vector multidimensional de parámetros. En visión por ordenador, trabajamos con imágenes y las convoluciones se aplican sobre más de un eje. Por ejemplo, si usamos una imagen (I) de dos dimensiones como entrada, también usaremos un filtro o *kernel* (K) de dos dimensiones y, dado que **la convolución cumple con la propiedad conmutativa**, podemos usar la siguiente formula:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i - m, j - n)K(m, n) \quad (3)$$

Figura 3. Ejemplo de *flipping* de un *kernel* que se usa antes de la operación de convolución

La propiedad conmutativa en la operación de convolución se cumple porque hacemos un *flipping* del *kernel* relativo a la entrada, como se muestra en la figura 3. No obstante, esta propiedad no es importante para la implementación de una red neuronal y es por ello que, para mejorar la eficiencia, la mayoría de librerías que trabajan con redes convolucionales implementan la función de *cross-correlation*, aunque por convención la llaman convolución:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n) \quad (4)$$

En resumen, por convención en el área de *deep learning* no nos importa mucho hacer el *flipping* del *kernel* y, técnicamente, lo que estamos haciendo se llama operación de *cross-correlation*. Sin embargo, la literatura que encontramos relacionada con el área de *deep learning* llama a esta operación «convolución», por convención. En este módulo didáctico usaremos esta misma convención para mantener la coherencia con otros textos. En otras áreas, como el procesamiento de la señal y otras especialidades matemáticas, hacer el *flipping* del *kernel* hace que se cumpla la propiedad asociativa, pero cuando trabajamos en *deep learning* esto realmente no es importante y nos permite simplificar el código de los modelos, mientras que las redes neuronales funcionan igual de bien.

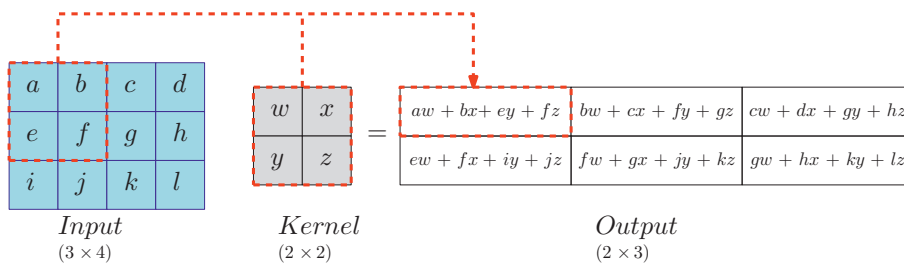
La propiedad conmutativa

$$f * g = g * f$$

La propiedad asociativa

$$(f * g) * h = f * (g * h)$$

La figura 4 muestra un ejemplo de convolución aplicada a un vector de datos de dos dimensiones, sin hacer *flipping* del *kernel*. La salida se muestra solamente en las posiciones donde el *kernel* cuadra con la imagen (no sale del borde). Se muestra un cuadrado y flechas para indicar como el elemento superior-izquierdo se forma aplicando el *kernel* a la parte correspondiente superior-izquierda de la región de la entrada.

Figura 4. Ejemplo de una convolución 2D sin *flipping* de *kernel*

Como se puede deducir del ejemplo presentado en la figura 4, las dimensiones de la salida de la operación de convolución se pueden calcular fácilmente a partir de las dimensiones de la entrada y el filtro aplicado. Siendo (I_1, I_2) las

dimensiones de la entrada y (K_1, K_2) las dimensiones del filtro, las dimensiones de la salida serán $(I_1 - K_1 + 1, I_2 - K_2 + 1)$.

La intuición detrás de esta operación es que la salida de la convolución obtendrá valores elevados únicamente cuando se detecte el patrón definido en el filtro. Por ejemplo, en la figura 5 podemos ver las tres primeras filas y columnas de una imagen de entrada, donde sus valores están normalizados en el rango $[0,1]$. En la misma imagen vemos que el filtro contiene valores elevados en la diagonal ascendente (desde el extremo inferior izquierdo hasta el extremo superior derecho), mientras que contiene valores negativos en el resto del filtro 3×3 . Por lo tanto, es evidente que este filtro busca detectar patrones de diagonales ascendentes en la imagen de entrada. Si sobre esta imagen aplicamos el filtro, podemos ver que el resultado es positivo, en concreto el valor de la convolución para esta primera celda es de 1.8.

Figura 5. Ejemplo de una convolución positiva

0.2	0.3	1.0	...
0.1	1.0	0.0	...
1.0	0.4	0.2	...
...	...	...	...

*

-1	-1	+1
-1	+1	-1
+1	-1	-1

=

1.8	...
...	...

Es importante remarcar que los valores negativos del filtro son importantes, ya que si hubiese ceros en estas posiciones indicaría que el valor que contengan es irrelevante. Por ejemplo, si suponemos que el filtro de este ejemplo contiene unos en la diagonal y ceros en el resto de celdas, entonces el resultado de la operación de convolución será 3.0. El problema es que si la imagen de entrada contiene unos en todas las posiciones indicadas de las tres primeras filas y columnas, entonces la convolución continuaría indicando un valor de 3.0, pero realmente la imagen de entrada no contiene un patrón de diagonal ascendente, sino simplemente una zona uniforme de valores altos (probablemente blanca en el caso de una imagen con un solo canal de color).

Figura 6. Ejemplo de una convolución negativa

1.0	0.2	0.2	...
0.1	1.0	0.0	...
0.3	0.4	1.0	...
...	...	...	...

*

-1	-1	+1
-1	+1	-1
+1	-1	-1

=

-1.2	...
...	...

Veamos ahora un ejemplo de aplicar el mismo filtro 3×3 en una imagen que contiene una diagonal descendente (es decir, desde el extremo superior izquierdo hasta el extremo inferior derecho). Como podemos ver en la figura 6, el resultado de la operación de convolución en este caso es un valor negativo, concretamente de -1.2.

Ambos resultados obtenidos en los ejemplos anteriores son coherentes con el concepto intuitivo que hemos comentado, ya que obtenemos valores altos cuando la presencia del patrón indicado en el filtro es muy clara en los datos de entrada y valores muy bajos (generalmente negativos) cuando no se detecta el patrón del filtro en los datos de entrada.

2.2. Ventajas derivadas de la convolución

La operación de convolución, descrita anteriormente, permite a las redes neuronales convolucionales una mejora en dos temas muy importantes para el entrenamiento de las redes:

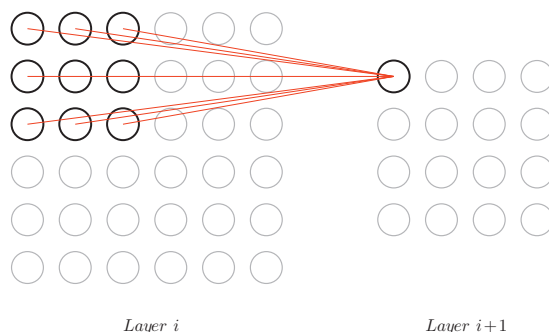
- 1) Interacciones o conexiones dispersas
- 2) Compartición de parámetros

En las siguientes secciones profundizaremos en estos dos conceptos claves de las redes neuronales convolucionales.

2.2.1. Interacciones o conexiones dispersas

Las redes neuronales tradicionales usan la multiplicación de matrices para describir la interacción entre cada unidad de entrada y cada unidad de salida. En este tipo de arquitectura, cada neurona de la red está conectada a todas las neuronas de las capas adyacentes (anterior y posterior). Este tipo de redes reciben el nombre de redes totalmente conectadas (*fully connected*). Las CNN usan un *kernel* mucho más pequeño que la entrada para trabajar con **conexiones dispersas** y las neuronas de una capa se conectan solo a un subconjunto de las neuronas de la capa siguiente y, asimismo, están conectadas solo a un subconjunto de neuronas contiguas de la capa anterior.

Figura 7. Ejemplo de interacciones dispersas de una red convolucional



La figura 7 muestra un ejemplo de esta arquitectura, donde cada grupo de 3×3 neuronas de la capa i se conectan con una neurona de la capa $i + 1$. En particular, el resto de los valores de los píxeles no tienen efecto sobre la salida, y a esto se refieren las **interacciones dispersas**. Esto implica que trabajemos

con menos parámetros, reduciendo a la vez los requerimientos de memoria y el tiempo de computación.

La estructura y el tamaño del conjunto de neuronas de la capa i que se conectan con una neurona de la capa $i + 1$ forma lo que se conoce como el **campo receptivo local** (*local receptive fields*). Por ejemplo, en el ejemplo anterior hemos visto un campo receptivo local de 3×3 .

2.2.2. Compartición de parámetros

En las redes totalmente conectadas, cada neurona oculta tiene un sesgo y un conjunto de pesos que depende del tamaño de su campo receptivo local. A diferencia de este tipo de capas, en las capas convolucionales se usan los mismos pesos y sesgos para todas las neuronas ocultas de una misma capa. En otras palabras, para una neurona oculta, su salida es:

$$\sigma \left(b + \sum_{l=0}^{n-1} \sum_{m=0}^{n-1} w_{(l,m)} a_{(j+l, k+m)} \right) \quad (5)$$

donde:

- σ representa la función de activación,
- b es el valor compartido de sesgo,
- $w_{(l,m)}$ es una matriz de $n \times n$ que contiene los pesos compartidos de las neuronas de una misma capa y
- $a_{(x,y)}$ es el valor de entrada de la posición (x, y) .

Esto significa que todas las neuronas de la capa oculta detectan exactamente la misma característica, solo que realizan esta función en diferentes ubicaciones de la imagen de entrada.

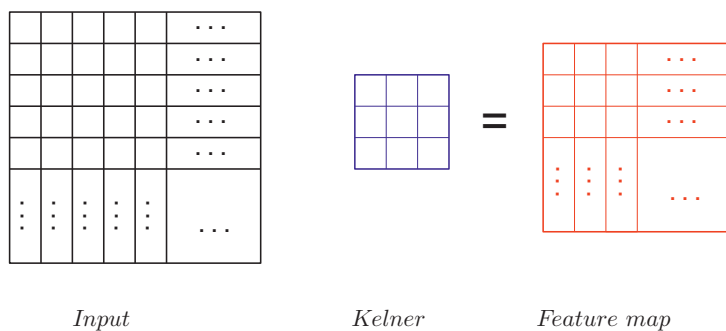
Supongamos que los pesos y sesgos son tales que la neurona oculta puede distinguir, digamos, un borde vertical en un campo receptivo local particular. Esta misma habilidad es probable que sea útil en otros lugares de la imagen. Por lo tanto, **es útil aplicar el mismo detector de características en todas partes de la imagen**. Para ponerlo en términos un poco más abstractos, las redes convolucionales están bien adaptadas a la invariancia de las características a detectar.

El mapa de características (en inglés, *feature map*) es el resultado de aplicar un determinado filtro a la correspondiente capa de entrada. Es decir, es el resultado de aplicar un determinado filtro a una capa de entrada concreta. Recibe el

nombre de «mapa de características» porque representa un mapa de donde se encuentra un determinado tipo de característica (**concretamente, la característica indicada en el filtro**) en los datos o imagen de entrada. Como ya hemos comentado anteriormente, las redes neuronales convolucionales buscan «características» como líneas rectas, bordes o incluso objetos.

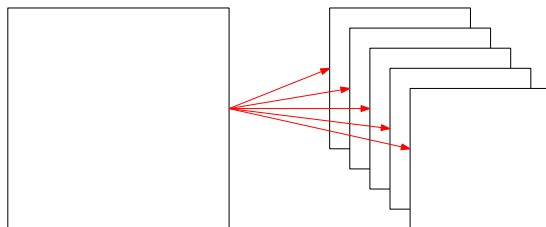
Intuitivamente, podemos pensar en un mapa de características como una matriz o un mapa, que nos proporciona información sobre la localización en los datos de entrada de la característica definida en el filtro, como se puede ver en la figura 8. Cada una de sus celdas contendrá valores altos o bajos, según si se ha detectado la característica indicada por el filtro en el campo receptivo local correspondiente.

Figura 8. Relación entre la entrada (*input*), el filtro (*kernel*) y el mapa de características (*feature map*)



Los pesos compartidos (*shared weights*) y el sesgo compartido son los pesos y el sesgo que definen el mapa de características. A menudo se dice que los pesos compartidos y el sesgo definen un *kernel* o filtro. Adicionalmente, los pesos y el sesgo compartidos pueden entenderse como los parámetros configurables que determinarán el mapa de características, que serán modificados y «aprendidos» por la red convolucional durante el proceso de entrenamiento.

Figura 9. Ejemplo de estructura con 5 mapas de características en la capa oculta



Según lo descrito anteriormente, es fácil darse cuenta de que las redes convolucionales deberán utilizar múltiples mapas de características según el número de patrones a detectar en los datos de entrada. Por ejemplo, la figura 9 muestra una estructura con una capa de entrada conectada a cinco mapas de características que forman la primera capa oculta de neuronas. Nótese que no se han representado en la imagen los cinco filtros necesarios para generar los mapas de características.

Una gran ventaja de compartir pesos y sesgos es que reduce en gran medida el número de parámetros involucrados en una red convolucional. Esto, a su vez, resultará en un entrenamiento más rápido para el modelo convolucional y, en última instancia, nos ayudará a construir redes profundas usando capas convolucionales.

2.3. Consideraciones finales

Empleando los mecanismos vistos anteriormente, las redes convolucionales tienen muchos menos parámetros que aprender. Este es un factor clave en el proceso de entrenamiento, ya que permite reducir el tiempo de entrenamiento y el tamaño del conjunto de datos necesario para «evitar» el sobreentrenamiento.

Otra ventaja importante de las redes convolucionales, que en este caso deriva de la compartición de parámetros, es una propiedad conocida como **representaciones equivalentes**. Esta propiedad se refiere al hecho de que la salida es invariante a los movimientos de traslación de las entradas. La estructura convolucional ayuda a las redes neuronales a codificar el hecho que una imagen desplazada (*shifted*) unos píxeles debe resultar en una estructura muy similar de características. Esta propiedad puede ser muy útil cuando nos preocupa la presencia de una determinada forma u objeto dentro de una imagen, no su posición. En este sentido, las redes convolucionales son bastante buenas capturando la invarianza en traslación.

En resumen, el bloque más importante de las CNN son las capas convolucionales. Las neuronas de la primera capa convolucional no están conectadas a cada píxel de la imagen de entrada, sino que solo lo están a los píxeles de sus respectivos campos receptivos locales. En consecuencia, cada neurona en la segunda capa convolucional está conectada solamente con las neuronas localizadas en un pequeño rectángulo de la primera capa.

Esta arquitectura permite a la red concentrarse en las características de bajo nivel en las primeras capas ocultas, para ensamblarlas luego en características de más alto nivel en la siguiente capa y así progresivamente. Esta estructura jerárquica es muy común en las imágenes del mundo real y es una de las razones por las cuales las CNN funcionan tan bien para resolver problemas de reconocimiento de imágenes.

3. Componentes de una red neuronal convolucional

Iniciaremos este capítulo centrándonos en los detalles de las capas convolucionales para entender mejor el funcionamiento de estas operaciones, junto con los parámetros relevantes para su correcto funcionamiento.

A continuación, veremos las principales capas que coexisten, junto con las capas convolucionales, en las CNN. Como veremos, las capas convolucionales son el núcleo de las CNN, pero es necesario añadir otro tipo de capas para obtener un buen rendimiento global de la red.

3.1. Capa de convolución

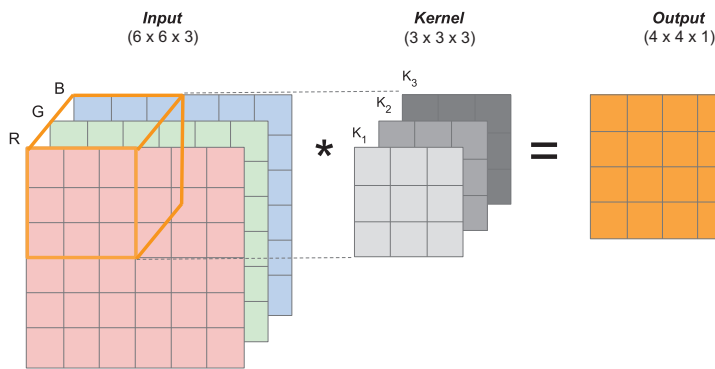
En esta sección nos centraremos en los detalles relevantes referentes a la implementación de las capas de convoluciones. En concreto, nos fijaremos en los siguientes conceptos relacionados con las capas de convolución:

- Filtros (*kernel*)
- Valor de relleno (*padding*)
- Convoluciones por pasos (*strided convolutions*)

3.1.1. Filtros o *kernel*

El **filtro**, también llamado ***kernel***, es la matriz de pesos de una capa de convolución dentro de una CNN. Implementa una convolución en toda la matriz de entrada. El *kernel*, normalmente, tiene un tamaño mucho más pequeño que la entrada.

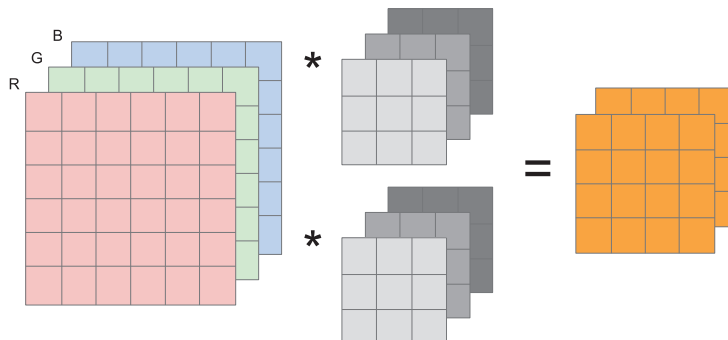
La entrada puede ser una imagen en escala de gris o en color, generalmente, RGB. Si es una imagen RGB, a la tercera dimensión se la llama *canal* o *volumen*. La representación más común es una imagen RGB donde cada canal es una matriz de dos dimensiones (2D) que representa un color. La figura 10 muestra un ejemplo donde se puede ver una entrada en RGB de 6×6 píxeles, a la cual se le aplica un filtro de 3×3 . Para calcular la salida se utiliza la ecuación 2 sobre las tres dimensiones o canales, agregando los resultados en un único valor según la siguiente expresión: $R * K_1 + G * K_2 + B * K_3$.

Figura 10. Convolución de un *kernel* sobre una imagen en tres dimensiones

Es importante notar que el *kernel* siempre tiene el mismo número de canales que la entrada. Es por ello que se produce una reducción de la dimensión (en todos los canales), excepto en el caso de emplear un *kernel* de 1×1 .

Generalmente, solemos tener diferentes *kernels* que capturan diferentes características, tal y como muestra la figura 11, donde hay dos *kernels*, cada uno con dimensión $3 \times 3 \times 3$.

Cabe destacar que, en este caso, el número de canales de la salida es dos, ya que este valor viene determinado por el número de *kernels* usados para la operación de convolución. Es decir, obtenemos una matriz de salida para cada filtro o *kernel*.

Figura 11. Convolución de dos *kernels* sobre una imagen en tres dimensiones

Podemos definir las dimensiones de la entrada como (n_w, n_h, n_{canal}) , siendo n_w el ancho (en píxeles) de la imagen, n_h la altura (en píxeles) de la imagen y n_{canal} el número de canales de la imagen. Cuando $n_{canal} = 1$, se trata de una entrada de dos dimensiones (2D), es decir, una imagen en escala de gris. De forma análoga, definimos las dimensiones del *kernel* como $(n_{k1}, n_{k2}, n_{canal})$. Aunque no es muy común, el *kernel* no tiene porque ser siempre cuadrado, pudiendo ser su dimensión $(n_{k1}, n_{k2}, n_{canal})$. Entonces, la dimensión de la salida será $(n_w - n_{k1} + 1, n_h - n_{k2} + 1, 1)$, y si tenemos n *kernels* diferentes, entonces la dimensión de la salida será de $(n_w - n_{k1} + 1, n_h - n_{k2} + 1, n)$.

3.1.2. Valor de relleno (*padding*)

Como ya hemos visto y comentado en este texto, la operación misma de convolución comporta una reducción de la dimensión de los datos de salida, respecto al tamaño de los datos de entrada. Generalmente, la dimensión de la salida de una capa de convolución suele ser más pequeña que la dimensión de la entrada. No obstante, queremos poder controlar este efecto en todo momento, ya que, por ejemplo, si queremos construir una red neuronal profunda (*deep*), no queremos que esto pase de forma muy rápida.

Por otro lado, si revisamos el procedimiento de la operación de convolución nos daremos cuenta de que los píxeles en posiciones del centro se usan con más frecuencia que los píxeles en las esquinas y los bordes. En consecuencia, la información de los bordes de las imágenes no se conserva del mismo modo que la información del centro tras la operación de convolución.

Estas dos consecuencias de la convolución pueden no ser deseables y el uso de valores de relleno (en inglés, *padding*) permite poder evitar ambos efectos en el proceso de convolución. El uso de valores de relleno es, simplemente, el proceso de agregar capas de ceros a los datos de entrada para evitar los problemas mencionados anteriormente.

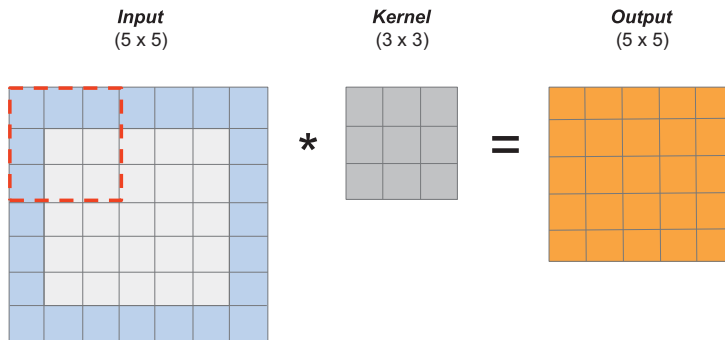
La figura 12 muestra un ejemplo de adición de valores de relleno (o *padding*) a una imagen de entrada, en este caso concreto de tamaño 5×5 . Las filas y columnas adicionales, que se muestran en color azul, son el resultado de aplicar el *padding*. Aunque en otros contextos puede ser diferente, en las redes neuronales convolucionales se suele aplicar siempre el *padding* con valores igual a 0, como se muestra en la imagen, ya que así no afecta al resultado de la convolución. En este caso, es decir, cuando se emplea el cero como valor de relleno, se denomina *zero padding* y es el que habitualmente emplearemos en redes convolucionales.

Figura 12. Representación de la aplicación de valores de relleno (*zero padding*) a una imagen de entrada, con un valor $p = 1$

0	0	0	0	0	0	0
0						0
0						0
0						0
0						0
0						0
0	0	0	0	0	0	0

Podemos ver un ejemplo de aplicación de *padding* en la figura 13. En este caso concreto, con un filtro de dimensión 3×3 , vemos que es suficiente con añadir un valor de relleno igual a 1 ($p = 1$) para mantener las dimensiones de la salida. De esta forma, la dimensión de la salida será la misma que la dimensión de la entrada, aun después de aplicar la convolución.

Figura 13. Ejemplo de aplicación de *zero padding* ($p = 1$) para mantener la dimensión de la salida. El borde exterior (cuadro marcado en azul) de la entrada representa las celdas con ceros añadidos



En general, las dimensiones de la salida de una convolución se pueden calcular a partir de los valores de relleno (p), de las dimensiones de entrada (w) y del tamaño del filtro (f). Asumiendo que la anchura y altura de la entrada son iguales, el tamaño de la salida queda determinado por la siguiente ecuación:

$$[(n_1 - f_1 + 2p) + 1] \times [(n_2 - f_2 + 2p) + 1] \quad (6)$$

Si queremos utilizar el *padding* para asegurarnos que la salida mantendrá el mismo tamaño que los datos de entrada, entonces podemos igualar la ecuación anterior al tamaño de la entrada y operar para obtener:

$$p = \left\lfloor \frac{(f - 1)}{2} \right\rfloor \quad (7)$$

donde asumimos que el filtro es cuadrado, i.e. $f_1 = f_2$, y el valor p debe ser un entero. Es importante notar que **únicamente es posible emplear el *padding* para mantener el tamaño de la entrada en una convolución si empleamos filtros de tamaño impar**, como por ejemplo 3×3 o 5×5 .

Retomando el ejemplo presentado en la figura 13, podemos ver que:

- Con un valor de relleno (*padding*) igual a 0 ($p = 0$), la ecuación anterior indica que la salida tendrá una dimensión de $(5 - 3 + 0) + 1 = 3$, lo cual se corresponde con la dimensión que tendría la salida sin emplear *padding*.
- Con un valor de relleno (*padding*) igual a 1 ($p = 1$), que se corresponde con el ejemplo presentado en la figura, la ecuación anterior indica un valor del tamaño de salida de $(5 - 3 + 2) + 1 = 5$, el cual permite mantener la dimensión de la salida.

3.1.3. Convoluciones por pasos

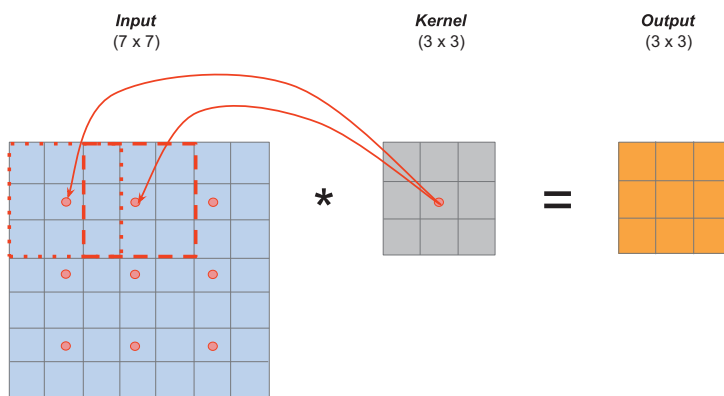
Como ya se ha mencionado, una de las ventajas de las redes convolucionales es que mediante la operación de convolución se reduce la dimensión de los datos de entrada y son más eficientes en los tiempos de ejecución. Usan-

do las **convoluciones por pasos** (*strided convolutions*, en inglés) podemos ser todavía mas eficientes, **aunque con el coste adicional de perder algunas características en la salida.**

Digamos que tenemos una entrada y un filtro y que en lugar de hacer la convolución de la forma tradicional (en cada celda de la entrada) lo hacemos con un paso = 2 (*stride* = 2). Esto quiere decir que **en lugar de aplicar la convolución en celdas consecutivas, la aplicaremos en las celdas con un salto de 2, tanto en la dimensión horizontal como en la vertical.**

La figura 14 muestra un ejemplo de convolución con *stride* = 2, marcando con un punto las celdas de la entrada donde se aplica la convolución. Como podemos ver, la imagen de entrada, con una dimensión de 7×7 se convierte a una salida de tamaño 3×3 , después de aplicarle un filtro de 3×3 con un valor de paso (*stride*) igual a 2.

Figura 14. Ejemplo de una convolución usando un *stride* de 2 ($s = 2$) y un *padding* de 0 ($p = 0$). Las celdas marcadas con un punto rojo de la entrada son aquellas donde se aplicará el *kernel* de convolución



Las dimensiones de la salida usando *stride* y *padding* vienen definidas por la siguiente formula:

$$\left\lfloor \frac{n_1 + 2p - f_1}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_2 + 2p - f_2}{s} + 1 \right\rfloor \quad (8)$$

donde n es la dimensión de la entrada, f la dimensión del filtro (o *kernel*), p el *padding* y s el paso (o *stride*).

Aunque las capas convolucionales son el factor clave de una red convolucional, estas no están formadas únicamente por capas convolucionales. Al contrario, encontramos una notable diversidad de capas de otros tipos que ayudan a generar la información deseada a partir de los datos de entrada.

A continuación, revisaremos las principales capas que pueden coexistir dentro de una red convolucional con las capas de convolución. En concreto, veremos cuatro tipos de capas:

- Capa de agrupamiento (*pooling*)
- Capa totalmente conectada (*fully-connected*)
- Capa de activación
- Capa de *dropout*

3.2. Capa de agrupamiento

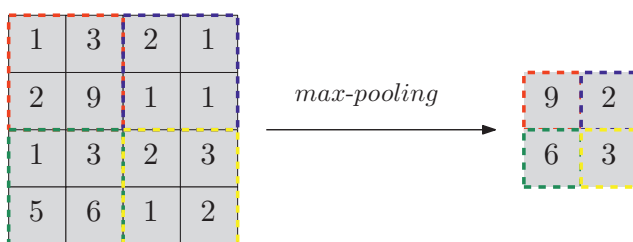
A parte de las capas convolucionales, las CNN usan a menudo **capas de agrupamiento** (*pooling*) con el doble propósito de hacer que algunas de las características sean más robustas y más eficientes.

Existen diferentes tipos de agrupamiento o *pooling*, aunque dos de los más empleados son:

- *Average-pooling*: aplica una operación de promedio (*average*) sobre todos los argumentos.
- *Max-pooling*: selecciona el valor máximo (*max*) sobre todos los argumentos.

Sin embargo, en el caso de las redes convolucionales, **el *max-pooling* es el más usado, porque simplemente selecciona el valor máximo del conjunto de valores de entrada.** El tamaño de la capa de agrupamiento determina el área (o campo receptivo local) donde se aplica la función de agrupamiento.

Figura 15. Ejemplo de agrupamiento basado en la función *max-pooling*



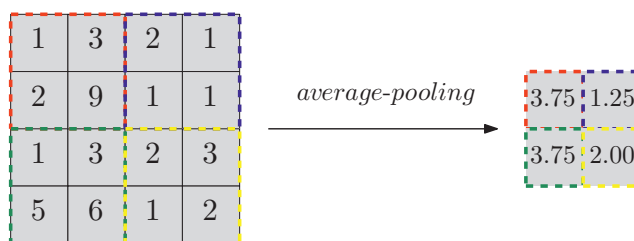
Imaginemos que tenemos una entrada de 4×4 y queremos aplicar una capa de agrupamiento basado en *max-pooling*, aplicando un filtro f de 2×2 y un *stride* igual a dos ($s = 2$), siendo estos los hiperparámetros de la capa de *max-pooling*. Entonces, la salida de esta capa tendrá una dimensión de 2×2 . Para ello, cogemos la entrada y lo dividimos en diferentes regiones, cuatro en este caso concreto. La figura 15 muestra la división de regiones diferenciadas por colores. Según lo comentado anteriormente, la salida se corresponde al máximo de cada región, ya que estamos aplicando la función *max-pooling*. En el ejemplo en cuestión, el máximo de la región superior-izquierda es 9 y así sucesivamente para las demás regiones.

La ecuación 8 nos sirve también aquí para calcular la dimensión de la salida en las capas de agrupamiento.

La intuición detrás de la operación de *max-pooling* en las redes convolucionales es que preservamos las características más frecuentes y más relevantes para cada cuadrante de la imagen. Una propiedad interesante de esta operación es que tiene un conjunto de hiperparámetros, pero estos no se tienen que aprender. Es decir, no es necesario que el algoritmo de entrenamiento aprenda estos parámetros, simplemente se fijan los valores de tamaño de filtro (f) y *stride* (s) en el momento de diseñar la red.

Si tenemos una entrada con un *canal* > 1 , las salidas tendrán el mismo número de canales. Por ejemplo, si tenemos una entrada de $5 \times 5 \times 2$ con $f = 3$ y $s = 1$, la salida será de $3 \times 3 \times 2$. Es decir, la operación de *max-pooling* se aplica de forma independiente sobre cada canal.

Figura 16. Ejemplo de agrupamiento basado en la función *average-pooling*



El otro tipo de operación de agrupamiento, aunque no se usa tan a menudo, es conocido como *average-pooling*. En este caso, en lugar de hacer el máximo sobre los valores de la región, hacemos el promedio (*average*) de todos los valores. En el ejemplo de la figura 16, podemos ver el mismo ejemplo comentado anteriormente para la función *max-pooling*, pero en este caso empleando la función *average-pooling*, donde el resultado sería $\{3.75; 1.25; 3.75; 2\}$.

Actualmente, la función *max-pooling* es la más utilizada en la mayoría de redes profundas, **excepto en redes neuronales muy profundas.**

3.3. Capa totalmente conectada

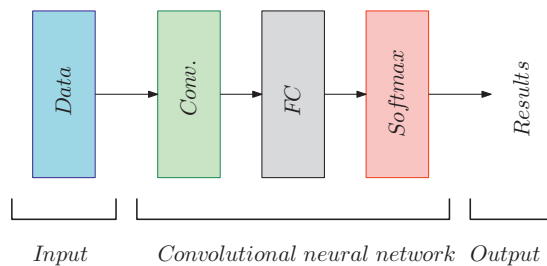
En general, una red neuronal convolucional está formada por tres grandes bloques o partes:

- 1) En primer lugar, se encuentra el bloque o parte convolucional, que incluye todas las capas de convolución, agrupamiento, activación, etc.
- 2) A continuación, en segundo lugar, suele aparecer un conjunto variable de capas totalmente conectadas (*fully connected layers*, FC). Estas capas, siguen los mismos patrones que hemos visto en las redes neuronales totalmente conec-

tadas, es decir, cada neurona de la capa i se conecta con todas las neuronas de la capa anterior, en este caso la capa $i - 1$, y a su vez, genera una salida que se conectará a todas las neuronas de la capa siguiente, es decir, la capa $i + 1$.

3) Finalmente, es habitual en problemas de clasificación que la última capa proporcione como salida un vector N dimensional, donde N es el número de clases que el modelo tiene que escoger. Por ejemplo, si queremos clasificar dígitos, N debería ser 10, ya que existen 10 dígitos diferentes, i.e. $\{0, 1, 2, \dots, 9\}$. Cada número de este vector N -dimensional representa una probabilidad de pertenecer a una clase. En el caso de emplear una función de salida *softmax*, el vector resultado para el modelo de clasificación de dígitos podría ser: $\{0; 0.1; 0.1; 0.75; 0; 0; 0; 0; 0; 0.05\}$, que representa un 10% que la imagen contenga un 1, un 10% que la imagen contenga un 2, un 75% de probabilidad que la imagen contenga un 3 y un 5% de probabilidad que la imagen sea un 9.

Figura 17. Estructura básica de una CNN para problemas de clasificación



La figura 17 muestra un esquema de la estructura comentada, con tres bloques o partes principales. Es importante remarcar que esta estructura básica se suele emplear en problemas de clasificación (muy similar en el caso de problemas de regresión), pero puede ser completamente diferente en el caso de otras tareas, como por ejemplo los modelos generativos.

Básicamente, la función principal de insertar una o varias capas totalmente conectadas al final de la red convolucional es determinar cuáles son las características de alto nivel que correlacionan con una clase en particular. Las capas convolucionales se centran en detectar patrones y características, mientras que las capas totalmente conectadas correlacionan dichas características con la salida del problema específico que intentamos resolver. Por eso es habitual emplear esta estructura de tres partes: una primera parte convolucional que va detectando patrones y características cada vez más complejos y abstractos (en general, cuanto más profunda la red, más capacidad de detectar patrones complejos y características abstractas); seguido de una segunda parte totalmente conectada que se focalizará en correlacionar estos patrones y características de alto nivel con el problema que estamos intentando resolver y, finalmente, una capa de salida, generalmente empleando una capa *softmax*, que nos proporcionará la salida del problema.

Entonces, durante el proceso de entrenamiento se deberán fijar estos pesos específicos, de forma que cuando se calcula el producto entre los pesos y la capa anterior, se obtienen las probabilidades correctas para cada clase. De esta forma, la primera capa *fully connected* recibe la salida de la capa anterior (que representan los mapas de activación (*feature maps*) de características de alto nivel) y determina cuáles son las características que correlacionan mejor con cada clase en particular.

Por ejemplo, si el objetivo es predecir si una imagen contiene un perro, tendrá valores altos en los mapas de activación que representen características de alto nivel como patas, cola, etc. De forma similar, si se quiere predecir si la imagen contiene un pájaro, tendrá valores altos en los mapas de activación que representen características de alto nivel como alas, pico, etc.

3.4. Capa de activación

Inmediatamente después de cada capa de convolución, es muy habitual aplicar una **capa no lineal** o **capa de activación**. El propósito de esta capa es introducir no linealidad al sistema, que básicamente ha estado realizando operaciones lineales durante las capas de convolución (multiplicaciones elemento a elemento y sumas). En este tipo de capas, se aplica la operación indicada sobre cada elemento de forma independiente. Por lo tanto, la dimensión de la salida siempre coincide con la dimensión de la entrada, dado que es una operación que se aplica elemento a elemento.

En el pasado, se usaban funciones no lineales como *tanh* o *sigmoid*, pero los investigadores han encontrado que la función ReLU funciona mucho mejor, ya que la red es capaz de entrenar más rápido sin que esto afecte al rendimiento final del modelo.

También ayuda a aliviar el problema de la desaparición del gradiente (*vanishing gradient*), que provoca que el entrenamiento sea muchísimo más lento en las primeras capas de la red que en las últimas, a causa del decrecimiento exponencial del gradiente a través de las capas.

Como hemos comentado, es habitual aplicar una función ReLU en estas capas de activación, lo que a veces se conoce simplemente como capa ReLU. La capa ReLU aplica la función $f(x) = \max(0, x)$ a todos los valores de la entrada. En términos básicos, esta capa solo cambia los valores negativos por 0, incrementando las propiedades no lineales del modelo y de toda la red, sin que esto afecte a los campos receptivos de la capa de convolución (Nair & Hinton, 2010).

Lectura complementaria

V. Nair y G. E. Hinton.
«Rectified linear units
improve restricted
boltzmann machines».
Proc. of Int. Conf. on
Machine Learning, Isreale,
2010

3.5. Capa de *dropout*

Las capas de *dropout* tienen una función muy específica en las redes neuronales: prevenir el sobreentrenamiento (*overfitting*). Esta capa desactiva (en inglés, *drops out*) un número aleatorio de entradas de la capa, poniéndolas a un valor igual 0.

El principal beneficio del *dropout* es que está forzando a la red a ser redundante, siendo esta capaz de dar la clasificación y las salidas correctas a pesar de que algunas entradas estén inactivas. De esta forma prevenimos que la red se ajuste demasiado al conjunto de datos de entrenamiento y ayuda a prevenir el problema de *overfitting*.

Es importante remarcar que, al igual que en las redes totalmente conectadas, esta capa se usa solamente durante el entrenamiento, pero no durante el proceso de test (Srivastava *et al.*, 2014).

Lectura complementaria

N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever y R. Salakhutdinov.
«Dropout: A Simple Way to Prevent Neural Networks from Overfitting». Journal of Machine Learning Research, Vol 15, p. 1929-1958, 2014

4. Estructura de una red neuronal convolucional

En este capítulo, veremos como a partir de las diferentes funciones, operaciones y capas que hemos visto anteriormente, podemos «agrupar» todas estas piezas para diseñar e implementar una red neuronal convolucional. Seguidamente, revisaremos un ejemplo completo de red neuronal convolucional y discutiremos su arquitectura básica y funcionamiento.

4.1. Estructura básica

En esta sección comentaremos algunas directrices o principios básicos para el diseño de redes neuronales convolucionales, aunque existe una enorme diversidad, no solamente de modelos y arquitecturas, sino también de problemas, dominios y tipos de datos. Es importante remarcar que no existe un único patrón de diseño, ya que estos cambian según el dominio o contexto en el que se aplique la red, y además evoluciona continuamente con una comunidad científica que avanza rápidamente en el diseño de nuevas y más potentes redes neuronales. Por lo tanto, esta sección no pretende ser una guía de diseño estructural de redes neuronales convolucionales. Simplemente, pretender dar algunos consejos o directrices básicos que se han empleado en el diseño de redes y que han sido válidos hasta ahora.

Como hemos comentado anteriormente, en general, podemos encontrar tres bloques o partes principales en una red neuronal convolucional, que son los siguientes:

- 1) Bloque o parte convolucional, que incluye todas las capas de convolución, agrupamiento, activación, etc.
- 2) Bloque o parte de capas totalmente conectadas (*fully connected layers*, FC). Estas capas, siguen los mismos patrones que hemos visto en las redes neuronales totalmente conectadas, es decir, cada neurona de la capa i se conecta con todas las neuronas de la capa anterior, en este caso la capa $i-1$, y, a su vez, genera una salida que se conectará a todas las neuronas de la capa siguiente, es decir, la capa $i+1$.
- 3) Bloque o parte de la capa de decisión.

Siguiendo esta estructura básica, en primer lugar encontramos las capas convolucionales, que son quienes tratarán directamente con los datos de entrada, generalmente en formato de imagen, ya sea en escala de grises o en color

RGB. Este es el bloque más extenso de la red, ya que en él se desarrollará todo el procesamiento relacionado con la identificación de patrones y características relevantes de la imagen, desde un nivel bajo en las primeras capas, hasta un nivel más complejo y abstracto en las últimas capas convolucionales. Los filtros de las primeras capas de la red detectan características de bajo nivel, tales como aristas y curvas. Como es de imaginar, para predecir si una imagen tiene algún tipo de objeto necesitamos que la red sea capaz de encontrar características de alto nivel, como pueden ser las manos, los ojos, las patas, etc.

Cuando hablamos de la primera capa, la entrada es la imagen original. No obstante, cuando hablamos de la segunda capa, la entrada es el mapa (o mapas) de activación que resultan de la primera capa. Cada capa de entrada, básicamente, está describiendo las localizaciones de la imagen original donde aparece determinadas características. Cuando aplicamos un conjunto de filtros sobre estos (pasada la segunda capa) la salida serán activaciones que se corresponden a características de más alto nivel. Algunas de estas características pueden ser semicírculos, cuadrados (combinaciones de varias líneas rectas) o polígonos (combinaciones de curvas y líneas rectas), entre muchas otras.

Por ejemplo, si tenemos una entrada de $32 \times 32 \times 3$, la salida de la red después de la primera capa de convolución, aplicando tres filtros de $5 \times 5 \times 3$ con *padding* igual a 0 y *stride* igual a 1, tendrá una dimensión de $28 \times 28 \times 3$. Si después pasamos por otra capa de convolución, la salida de la primera capa pasa a ser la entrada de la segunda y esta es un poco más difícil de visualizar. En este sentido, la figura 18 muestra la visualización de las características (Zeiler & Fergus, 2014) de una red convolucional similar a ImageNet (Deng *et al.*, 2009). Se muestran los patrones de las *top 9* activaciones de los mapas de características del conjunto de datos de validación. Son patrones reconstruidos de los datos de validación que causan activaciones altas en los mapas de características.

Figura 18. Ejemplo de filtros en diferentes capas de una red convolucional



Fuente: (Zeiler & Fergus, 2014).

Otra característica importante es que, a medida que vamos más y más profundo en la red, los filtros tienen un campo receptivo más grande. Es decir, consideran información de una área más grande de la imagen original.

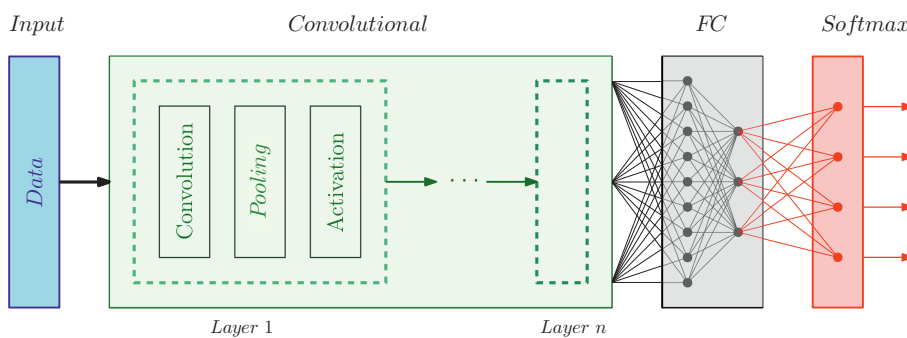
Lectura complementaria

J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li y L. Fei-Fei. «ImageNet: A Large-Scale Hierarchical Image Database». CVPR09, 2009.

Lectura complementaria

M. D. Zeiler y R. Fergus. «Visualizing and understanding convolutional networks». Proc. of the Eur. Conf. on Computer Vision, p. 818-833, Europe, 2014.

Figura 19. Esquema de la estructura básica de una CNN



La figura 19 muestra un esquema de la estructura comentada. Como podemos apreciar, el bloque convolucional está formado por múltiples «capas» convolucionales. Aunque *a priori* podamos entender que una capa de convolución es una capa que realiza la operación de convolución, tal como la hemos descrito en este texto, por convenio y para simplificar la notación de las redes neuronales profundas, se entiende que una capa convolucional implica tres capas que suelen trabajar siempre juntas, una tras otra, que son: una capa convolucional, tal como hemos descrito en este texto, más una capa de agrupamiento (*pooling*) y una capa de activación (generalmente, ReLU).

A medida que avanzamos en la red y vamos aplicando más y más capas de convolución, obtenemos mapas de activación que representan características más complejas y abstractas. Al final de la red, podemos tener filtros que se activan cuando hay un determinado dígito en la imagen, filtros que se activan cuando hay objetos de color verde, etc.

Por otra parte, a medida que avanzamos en la red, nos encontramos que la dimensión de los datos se va reduciendo, debido principalmente a las operaciones de convolución, agrupamiento (*pooling*) y otras. Sin embargo, normalmente esta reducción de la dimensión de los datos va acompañada de un aumento en el volumen de los mismos, debido al uso de múltiples filtros (*kernels*) en las operaciones de convolución. Por lo tanto, podemos decir que, en general, los datos se van reduciendo en sus dos primeras dimensiones, mientras que aumentan en su tercera dimensión, conforme profundizamos en la red neuronal convolucional.

La última capa convolucional, cualquiera que sea su dimensión, se suele «aplanar» (*flatten*, en inglés) para colocar todos los datos en formato de vector unidimensional. Este vector será el punto de entrada al bloque totalmente conectado (etiquetado como *FC* en la figura 19). A partir de este punto, tenemos una estructura de red neuronal totalmente conectada, donde todas las neuronas de una capa i se conectan con todas las neuronas de la capa siguiente ($i + 1$) y reciben sus entradas de la capa $i - 1$.

Este bloque, como la mayoría de redes neuronales completamente conectadas, sigue una forma de «embudo». Es decir, generalmente, el número de neuronas de cada capa oculta disminuye conforme avanzamos en la red.

Finalmente, la salida de la última capa de neuronas del bloque *FC* se conecta a la capa de salida, que generalmente es una capa *softmax* en el caso de problemas de clasificación.

4.2. Ejemplo de red neuronal convolucional

Imaginemos que queremos resolver el problema de reconocimiento de dígitos en una imagen y tenemos una entrada de $32 \times 32 \times 3$, i.e. una imagen RGB de 32×32 píxeles. Vamos a crear una red neuronal inspirada en LeNet-5 (Bengio *et al.*, 1998) para resolver este problema. El ejemplo se puede ver en la figura 20.

En primer lugar, vamos a usar 6 filtros de 5×5 , un *stride* de 1 y sin *padding* en la primera capa. Utilizamos los 6 filtros teniendo en consideración el sesgo (*bias*) y aplicamos no linealidad mediante una capa de activación ReLU. A esta capa la llamaremos *conv1*, y su salida presenta un tamaño de $28 \times 28 \times 6$. Seguidamente aplicamos una capa de agrupación empleando la función *max-pooling* con unos valores de tamaño de filtro de 2×2 ($f = 2$) y un valor de paso (*stride*) igual a 2 ($s = 2$). Si no se menciona el uso de valores de relleno (*padding*), asumimos que este es 0. Este proceso reduce el alto y ancho de la imagen en un factor de 2, obteniendo una salida de $14 \times 14 \times 6$. El número de canales se mantiene igual, es decir, con un total de 6 canales. Nos referimos a esta salida como *pool1*.

En la literatura de las redes neuronales convolucionales, existe una convención en referencia a lo que llamamos «capa», debido a que una capa de convolución suele ir seguida de una capa de agrupamiento: una capa de convolución más una capa de agrupamiento, forman una sola «capa» de la red neuronal. En el ejemplo que estamos viendo, la *conv1* más *pool1* forman la capa 1 (*layer 1*) de la red.

Cuando se reporta el número de capas de una red neuronal, normalmente solo se reporta el número de capas que tiene pesos, es decir, que tiene parámetros. Dado que la capa de agrupamiento no tiene pesos (solo hiperparámetros), no se reporta cuando mencionamos el número de capas que tiene la red.

Volviendo al ejemplo, dado el volumen de entrada de $14 \times 14 \times 6$, aplicamos otra capa convolucional. Esta vez con 16 filtros de 5×5 y paso igual a 1 ($s = 1$). La salida la hemos etiquetado con el nombre *conv2*, y presenta una dimensión de $10 \times 10 \times 16$. Aplicamos, a continuación, una función *max-pooling* con tamaño de filtro 2×2 ($f = 2$) y paso igual a 2 ($s = 2$), obtenien-

Lectura complementaria

Y. Bengio, Y. LeCun, L. Bottou y P. Haffner. «Gradient based learning applied to document recognition». Proc. IEEE, 1998.

MNIST

La base de datos MNIST es una gran base de datos de dígitos escritos a mano que se usa comúnmente para entrenar varios sistemas de procesamiento de imágenes.

do una salida de $5 \times 5 \times 16$, a la que llamamos *pool2*. Estas dos capas forman la capa 2 (*layer 2*) de la red neuronal.

En la parte final de esta red vamos a incorporar las capas totalmente conectadas (*fully-connected*). En primer lugar, debemos reestructurar la salida de *pool2* en forma de vector unidimensional, obteniendo un vector de 400×1 (a partir del vector multidimensional de $5 \times 5 \times 16$). Esta capa es, simplemente, una reestructuración de la dimensión de los datos, pero no realiza ninguna operación sobre los mismos.

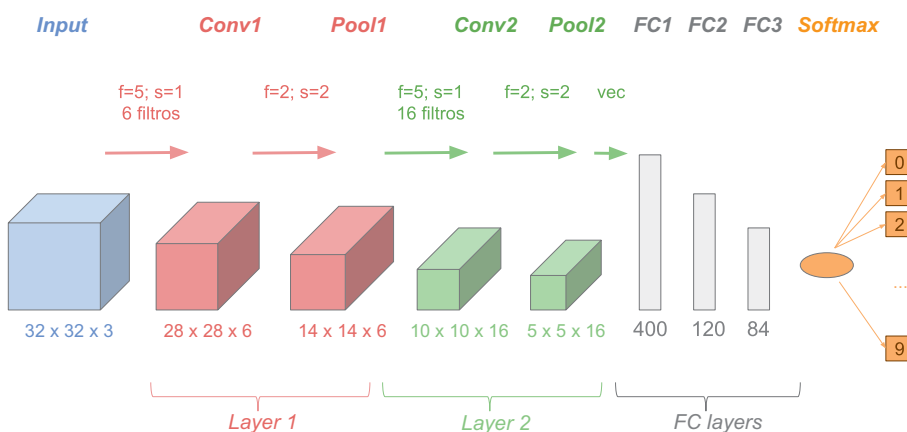
A continuación, lo que vamos a hacer es conectar estas 400 unidades, estas 400 neuronas, con la siguiente capa de 120 unidades. Esto es la primera capa totalmente conectada que llamaremos *FC2*, donde tenemos 400 unidades conectadas densamente con las 120 unidades, es decir, cada una de las 400 unidades está conectada con cada una de las 120 unidades, lo que genera un total de 48.120 parámetros que la red deberá aprender durante el proceso de entrenamiento.

Finalmente añadimos otra capa de 84 unidades, a la que llamaremos *FC3*, obteniendo 84 unidades que serán la entrada de una capa *softmax*. En este caso, la matriz que deberá aprender la red tendrá un tamaño de 10.164 parámetros. Si lo que queremos hacer es reconocimiento de dígitos, esta será una *softmax* con 10 salidas, i.e. $\{0, 1, 2, \dots, 9\}$, lo que implica una última matriz de 850 parámetros que deberemos fijar durante el proceso de entrenamiento.

Lectura complementaria

J. S. Denker, R. E. Howard, W. Hubbard, Y. LeCun, B. Boser y L. D. Jackel. «Backpropagation applied to handwritten zip code recognition». *Neural Computation*, Vol. 4, p. 541-551, 1989.

Figura 20. Ejemplo de una CNN para detectar dígitos inspirada en la arquitectura LeNet-5



Fuente: (Denker et al., 1989).

Una recomendación o guía para el diseño y creación de redes convolucionales es no intentar inventar una arquitectura y configuración de hiperparámetros desde cero, si no investigar en la literatura y seleccionar una arquitectura que funcione bien para aplicaciones similares y, luego, intentar afinar la red para mejorar los resultados con los datos específicos del problema a resolver.

Si nos fijamos en el ejemplo de la figura 20, observamos que a medida que vamos profundizando en la red neuronal, el ancho y alto de los volúmenes se hace más pequeños (de 32 a 5) mientras que el número de canales se incrementa (de 3 a 16). Vemos que las capas de convolución (*conv*) van seguidas de una capa de agrupamiento (*pool*) y las capas *fully-connected* siempre se encuentran en la parte final de la red, seguidas de una capa *softmax* (en los problemas de clasificación). Este es el patrón típico y estándar que hemos comentado para arquitecturas de redes neuronales convolucionales en problemas de clasificación.

La tabla 1 muestra los valores de las capas de activación, sus medidas y el número de parámetros de la red. Para calcular el número de parámetros se usan las siguientes formulas:

- **Capa de entrada** (*input layer*): lo que hace la capa de entrada es leer la imagen y no hay parámetros que aprender.
- **Capas convolucionales** (*Convolutional layers*): consideramos una capa convolucional que tiene l canales de entrada, k canales de salida y un tamaño del filtro de f . El número total de pesos será igual a $f \times f \times l \times k$. Adicionalmente, también debemos considerar el valor del sesgo (*bias*) para cada mapa de características, por lo tanto, el número total de parámetros será de $((f \times f \times l) + 1) \times k$.
- **Capas de agrupamiento** (*pooling layers*): no hay parámetros que aprender durante el proceso de entrenamiento de la red.
- **Capas totalmente conectadas** (*fully-connected layers*): en este tipo de capas, cada unidad de entrada tiene pesos separados en cada unidad de salida. Para n entradas y m salidas, el número de pesos es $n \times m$. También tenemos el sesgo para cada neurona de salida, siendo por tanto el número total de parámetros igual a $(n + 1) \times m$.
- **Capa de salida** (*output layer*): la capa de salida suele ser una capa *fully-connected*, donde el número de parámetros será $(n + 1) \times m$.

Tabla 1. Valores y número de parámetros de la red convolucional del ejemplo de la figura 20

Capa	Dimensión	Tamaño	Parámetros
<i>Input</i>	$(32 \times 32 \times 3)$	3.072	0
<i>Conv1</i>	$(28 \times 28 \times 6)$	4.704	456
<i>Pool1</i>	$(14 \times 14 \times 6)$	1.176	0
<i>Conv2</i>	$(10 \times 10 \times 16)$	1.600	2.416
<i>Pool2</i>	$(5 \times 5 \times 16)$	400	0
<i>FC2</i>	(120×1)	120	48.120
<i>FC3</i>	(84×1)	84	10.164
<i>Softmax</i>	(10×1)	10	850

Un último comentario en referencia a las redes convolucionales y el número de parámetros que estas deben aprender durante el proceso de entrenamiento:

- 1) Las capas de convolución tienen un número de parámetros relativamente pequeño a aprender, mientras que la mayoría de parámetros que estas redes deben aprender provienen de las capas *fully-connected*.
- 2) El tamaño de las activaciones tiende a disminuir a medida que vamos profundizando en la red, no obstante si disminuye de forma muy brusca no es una buena señal cuando buscamos obtener una buena precisión en las predicciones de la red neuronal.

Son muchos los investigadores que han estudiado cual es la mejor forma de poner las diferentes piezas de las redes convolucionales, de tal forma que se pueda maximizar la efectividad de las redes neuronales.

Resumen

Hemos iniciado este módulo didáctico presentando una breve discusión sobre las redes neuronales convolucionales y algunas de sus aplicaciones «clásicas» en dominios o problemáticas donde las redes neuronales convolucionales se aplicaron inicialmente.

En la segundo capítulo de este texto, hemos introducido la operación de convolución, que es el núcleo de las redes neuronales convolucionales. Esta operación es básica para poder entender las redes convolucionales, así como sus ventajas y sus posibles limitaciones. Como hemos visto en el siguiente capítulo, las capas convolucionales están formadas por neuronas que aplican la operación de convolución y son la base de las redes neuronales convolucionales. Sin embargo, no son el único tipo de capa que aparece en este tipo de redes. Es muy habitual y, generalmente necesario, el empleo de capas de agrupamiento, capas de activación, capas completamente conectadas y capas de *dropout* para que la red pueda efectuar las operaciones deseadas y conseguir un buen comportamiento en diferentes entornos o dominios.

Finalmente, en el cuarto y último capítulo, hemos analizado la estructura básica de una red neuronal convolucional y discutido un ejemplo concreto de una red inspirada en LeNet-5 y desarrollada para el reconocimiento de dígitos escritos manualmente.

Glosario

aprendizaje automático *m* El aprendizaje automático (más conocido por su denominación en inglés, *machine learning*) es el conjunto de técnicas, métodos y algoritmos que permiten a una máquina aprender de manera automática en base a experiencias pasadas.

convolución *f* La convolución es una operación matemática sobre dos funciones (f y g) que produce una tercera función ($s = f * g$) que expresa como la forma de una es modificada por la otra.

MNIST *f* La base de datos MNIST es una gran base de datos de dígitos escritos a mano que se usa comúnmente para entrenar varios sistemas de procesamiento de imágenes.

propiedad asociativa *f* Es una propiedad que puede tener una operación matemática $\odot$ según la cual, dados tres o más elementos cualquiera de un conjunto determinado, se verifica que dicha operación cumple la igualdad $(f \odot g) \odot h = f \odot (g \odot h)$.

propiedad conmutativa *f* Es una propiedad que puede tener una operación matemática $\odot$ según la cual el resultado de operar dos elementos no depende del orden en el que se toman. Es decir, la operación $\odot$ cumple la propiedad conmutativa si, y solamente si, $f \odot g = g \odot f$.

RGB *m* El modelo RGB (de las siglas en inglés: *Red*, *Green* y *Blue*) es un modelo de color basado en la síntesis aditiva con el que es posible representar un color mediante la mezcla por adición de los tres colores de luz primarios.

Bibliografía

Bengio, Y., Goodfellow, I., Courville, A. (2016). *Deep Learning*. Cambridge, MA: MIT Press.

Casas Roma, J., Lozano Bagén, T., Bosch Rué, A. (2020). *Deep learning: Principios y fundamentos*. Barcelona: Editorial UOC.

Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 2ª edición. O'Reilly Media, Inc.

Haykin, S. O. (2009). *Neural Networks and Learning Machines*. 3ª edición. Pearson.

Nielsen, M. A. (2015). *Neural Networks and Deep Learning*. Determination Press.